

Quiz Master App Documentation

Author

Name: Astitva Vats Pandey

Roll No: 21f3003255

Student Email: 21f3003255@ds.study.iitm.ac.in

I am a student at IIT Madras pursuing Data Science and Programming. This project reflects my ability to design, develop, and deploy a functional quiz management system using Flask and related technologies.

Description

This project, "Quiz Master," is a web-based quiz management system that allows administrators to create quizzes, manage questions, and track user performance. Users can attempt quizzes, view their results, and track their subject-wise performance through visual charts.

Technologies Used

- **Flask:** Web framework to handle the backend logic
 - **Flask-SQLAlchemy:** ORM for managing the database
 - **Flask-Login:** Authentication and user management
 - **SQLite:** Lightweight database for storing quizzes, users, and scores
 - **Jinja2:** Templating engine for rendering dynamic pages
 - **Chart.js:** For visualizing user performance
 - **Bootstrap & CSS:** For styling the frontend
-

DB Schema Design

The database consists of the multiple related tables:

- Each subject contains multiple chapters
- Each chapter is related to multiple quizzes
- Each quiz has multiple question
- Questions and Scores are stored in different separate table

Database Tables

1. **Users** (user_id, username, password, role) - Stores user credentials and roles.
2. **Quiz** (quiz_id, name, subject_name, total_marks, date_of_quiz) - Stores quiz details.
3. **Questions** (question_id, quiz_id, question_statement, option1, option2, option3, option4, correct_option, marks) - Stores questions linked to quizzes.
4. **Scores** (score_id, user_id, quiz_id, total_scored) - Stores user scores for quizzes.
5. **Chapter** (id, name, description, subject_id, subject_name) - Stores chapters for subjects
6. **Subject** (id, name, description) - Stores subject details

This relational structure ensures data integrity and efficient retrieval of information.

Architecture and Features

The project follows the **MVC (Model-View-Controller)** architecture:

- **Models:** Defined in `models/models.py` for database interactions.
- **Views:** HTML templates using Jinja2 in the `templates/` folder and CSS and surface level Javascript for styling in `static/` folder
- **Controllers:** Defined in `controllers/` folder to handle routing and logic.

Implemented Features:

- **Admin Panel:**
 - CREATE, UPDATE, DELETE Subjects, Chapters and related Quizzes
 - Monitor user performance.
 - Search chapters, quizzes and users using keywords

- **User Dashboard:**
 - Start and complete quizzes.
 - View quiz details and schedule
 - View past quiz scores, track progress.
 - Subject wise performance analytics using Chart.js.
 - Search for quizzes using keywords.
 - **Authentication:** User login and role-based access control.
-

Video Demonstration

This project successfully integrates database management, web development, and data visualization, making it a complete quiz platform.

 QuizMasterDemo.mp4